spark basics

Sam · 18 July

Week 5 - Apache Spark (PySpark) | Data Cleaning, Transformation & Aggregation

In [1]:
```
import sys
print(sys.executable)
```

Out [1]:
```
d:\Samradni Dahiphale\Projects\Celebal task\Data_Engineering_Internship_Celebal_Technologies\Assessment 5\.venv\Scripts\python.exe
```

In [2]:
```
import os

os.environ["HADOOP_HOME"] = r"C:\hadoop"
os.environ["hadoop.home.dir"] = r"C:\hadoop"
```

In [3]:
```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = (
    SparkSession.builder
    .master("local[*]")
    .appName("Celebal-Week5-Spark-Assignment")
    .getOrCreate()
)

print("Spark Started")
print("Spark Version:", spark.version)
```

Out [3]:
```
d:\Samradni Dahiphale\Projects\Celebal task\Data_Engineering_Internship_Celebal_Technologies\Assessment 5\.venv\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
Spark Started
Spark Version: 4.2.0
```

In [4]:
```
print("spark" in globals())
```

Out [4]:
```
True
```

This notebook loads the Kaggle Employee dataset placed in the repo under `Employee/Employee.csv` (as you mentioned: `Employee/Employee.csv`).

It performs: duplicates removal, null handling (fillna), schema casting, filtering, column renaming, aggregation, groupBy, and saves results to `output/`.

In [5]:
```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName('Celebal-Week5-Spark-Assignment').getOrCreate()

print('Spark Started')
print('Spark version:', spark.version)
```

Out [5]:
```
Spark Started
Spark version: 4.2.0
```

In [6]:

```
# --- Load CSV ---
# Dataset location: top-level `Employee/Employee.csv`
input_path = 'D:\Samradni Dahiphale\Projects\Celebal task\Data_Engineering_Internship_Celebal_Technologies\Assessment 5\Employee.c

df = (
    spark.read
    .option('header', True)
    .option('inferSchema', True)
    .csv(input_path)
)

print('Loaded rows:', df.count())
print('Columns:', df.columns)
df.show(5, truncate=False)
df.printSchema()
```

Out [6]:

```
<>:3: SyntaxWarning: "\S" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\S"? A raw string
is also an option.
<>:3: SyntaxWarning: "\S" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\S"? A raw string
is also an option.
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_37964\1402971821.py:3: SyntaxWarning: "\S" is an invalid escape sequence. Such sequences
will not work in the future. Did you mean "\\S"? A raw string is also an option.
  input_path = 'D:\Samradni Dahiphale\Projects\Celebal task\Data_Engineering_Internship_Celebal_Technologies\Assessment
5\Employee.csv'
Loaded rows: 4653
Columns: ['Education', 'JoiningYear', 'City', 'PaymentTier', 'Age', 'Gender', 'EverBenched', 'ExperienceInCurrentDomain',
'LeaveOrNot']
+---------+-----------+---------+-----------+---+------+-----------+------------------------+----------+
|Education|JoiningYear|City     |PaymentTier|Age|Gender|EverBenched|ExperienceInCurrentDomain|LeaveOrNot|
+---------+-----------+---------+-----------+---+------+-----------+------------------------+----------+
|Bachelors|2017       |Bangalore|3          |34 |Male  |No         |0                       |0         |
|Bachelors|2013       |Pune     |1          |28 |Female|No         |3                       |1         |
|Bachelors|2014       |New Delhi|3          |38 |Female|No         |2                       |0         |
|Masters  |2016       |Bangalore|3          |27 |Male  |No         |5                       |1         |
|Masters  |2017       |Pune     |3          |24 |Male  |Yes        |2                       |1         |
+---------+-----------+---------+-----------+---+------+-----------+------------------------+----------+
only showing top 5 rows
root
 |-- Education: string (nullable = true)
 |-- JoiningYear: integer (nullable = true)
 |-- City: string (nullable = true)
 |-- PaymentTier: integer (nullable = true)
 |-- Age: integer (nullable = true)
 |-- Gender: string (nullable = true)
 |-- EverBenched: string (nullable = true)
 |-- ExperienceInCurrentDomain: integer (nullable = true)
 |-- LeaveOrNot: integer (nullable = true)
```

In [7]:

```
# --- Data Cleaning: Remove duplicates ---
df_clean = df.dropDuplicates()
print('Rows after dropDuplicates():', df_clean.count())
```

Out [7]:

```
Rows after dropDuplicates(): 2764
```

In [8]:

```
# --- Data Cleaning: Handle nulls (fillna) ---
# Strategy: fill numeric nulls with 0 and string nulls with 'Unknown'.

numeric_cols = [t[0] for t in df_clean.dtypes if t[1] in ('int', 'bigint', 'double', 'float', 'decimal')]
string_cols = [t[0] for t in df_clean.dtypes if t[1] == 'string']

fill_map = {c: 0 for c in numeric_cols}
fill_map.update({c: 'Unknown' for c in string_cols})

df_clean = df_clean.fillna(fill_map)
```

```
null_counts = df_clean.select([F.sum(F.col(c).isNull().cast('int')).alias(c) for c in df_clean.columns])
null_counts.show(truncate=False)
```

Out [8]:

```
+---------+-----------+----+-----------+---+------+-----------+-----------------------+----------+
|Education|JoiningYear|City|PaymentTier|Age|Gender|EverBenched|ExperienceInCurrentDomain|LeaveOrNot|
+---------+-----------+----+-----------+---+------+-----------+-----------------------+----------+
|0        |0          |0   |0          |0  |0     |0          |0                      |0         |
+---------+-----------+----+-----------+---+------+-----------+-----------------------+----------+
```

In [9]:

```
# --- Schema fixes (casting) ---
# Common numeric columns in employee datasets (adjust if needed).
likely_numeric = ['Age', 'Salary', 'MonthlyIncome', 'YearsAtCompany', 'YearsExperience']
for c in likely_numeric:
    if c in df_clean.columns:
        df_clean = df_clean.withColumn(c, F.col(c).cast('double'))

print('Post-cast schema:')
df_clean.printSchema()
```

Out [9]:

```
Post-cast schema:
root
 |-- Education: string (nullable = false)
 |-- JoiningYear: integer (nullable = false)
 |-- City: string (nullable = false)
 |-- PaymentTier: integer (nullable = false)
 |-- Age: double (nullable = false)
 |-- Gender: string (nullable = false)
 |-- EverBenched: string (nullable = false)
 |-- ExperienceInCurrentDomain: integer (nullable = false)
 |-- LeaveOrNot: integer (nullable = false)
```

In [10]:

```
# --- Filtering ---
filtered = df_clean

if 'Age' in df_clean.columns:
    filtered = filtered.filter(F.col('Age') >= 25)

# Choose the first available categorical filter column.
cat_col = None
for c in ['Department', 'Region', 'Category', 'JobRole']:
    if c in df_clean.columns:
        cat_col = c
        break

if cat_col:
    filtered = filtered.filter(F.col(cat_col).isNotNull())

filtered.show(10, truncate=False)
print('Filtered rows:', filtered.count())
```

Out [10]:

```
+---------+-----------+---------+-----------+----+------+-----------+-----------------------+----------+
|Education|JoiningYear|City     |PaymentTier|Age |Gender|EverBenched|ExperienceInCurrentDomain|LeaveOrNot|
+---------+-----------+---------+-----------+----+------+-----------+-----------------------+----------+
|PHD      |2012       |New Delhi|3          |27.0|Male  |No         |5                      |0         |
|PHD      |2018       |Bangalore|3          |26.0|Male  |No         |4                      |1         |
|Bachelors|2013       |Bangalore|3          |28.0|Female|No         |4                      |1         |
|Bachelors|2018       |Bangalore|3          |26.0|Male  |Yes        |4                      |1         |
|Masters  |2015       |New Delhi|3          |29.0|Male  |No         |2                      |0         |
|Bachelors|2012       |Bangalore|3          |29.0|Male  |No         |2                      |1         |
|Bachelors|2016       |Bangalore|3          |28.0|Male  |Yes        |2                      |0         |
|PHD      |2018       |New Delhi|2          |30.0|Female|No         |4                      |1         |
|Bachelors|2015       |Pune     |3          |28.0|Male  |No         |5                      |0         |
|Bachelors|2015       |Pune     |3          |29.0|Male  |No         |0                      |0         |
+---------+-----------+---------+-----------+----+------+-----------+-----------------------+----------+
```

```
only showing top 10 rows
Filtered rows: 2531
```

In [11]:

```
# --- Transformations: rename columns (example) ---
# Rename Salary -> MonthlySalary if applicable
if 'Salary' in filtered.columns and 'MonthlySalary' not in filtered.columns:
    filtered = filtered.withColumnRenamed('Salary', 'MonthlySalary')

# If Age exists and you want a cleaner name
if 'Age' in filtered.columns and 'EmployeeAge' not in filtered.columns:
    filtered = filtered.withColumnRenamed('Age', 'EmployeeAge')

filtered.show(5, truncate=False)
```

Out [11]:

```
+---------+-----------+---------+-----------+-----------+------+-----------+------------------------+----------+
|Education|JoiningYear|City     |PaymentTier|EmployeeAge|Gender|EverBenched|ExperienceInCurrentDomain|LeaveOrNot|
+---------+-----------+---------+-----------+-----------+------+-----------+------------------------+----------+
|PHD      |2012       |New Delhi|3          |27.0       |Male  |No         |5                       |0         |
|PHD      |2018       |Bangalore|3          |26.0       |Male  |No         |4                       |1         |
|Bachelors|2013       |Bangalore|3          |28.0       |Female|No         |4                       |1         |
|Bachelors|2018       |Bangalore|3          |26.0       |Male  |Yes        |4                       |1         |
|Masters  |2015       |New Delhi|3          |29.0       |Male  |No         |2                       |0         |
+---------+-----------+---------+-----------+-----------+------+-----------+------------------------+----------+
only showing top 5 rows
```

In [12]:

```
# --- Basic Aggregations ---
print('Total (filtered) employees:', filtered.count())

# Choose a salary-like numeric column
salary_col = None
for c in ['MonthlySalary', 'Salary', 'MonthlyIncome']:
    if c in filtered.columns:
        salary_col = c
        break

if salary_col:
    filtered.select(
        F.count(F.lit(1)).alias('row_count'),
        F.avg(F.col(salary_col)).alias('avg_salary'),
        F.min(F.col(salary_col)).alias('min_salary'),
        F.max(F.col(salary_col)).alias('max_salary'),
        F.sum(F.col(salary_col)).alias('sum_salary')
    ).show(truncate=False)
else:
    print('No Salary-like column found for min/max/avg/sum aggregation.')
```

Out [12]:

```
Total (filtered) employees: 2531
No Salary-like column found for min/max/avg/sum aggregation.
```

In [13]:

```
# --- Group By Aggregations ---

group_col = None
for c in ['Department', 'Region', 'Category', 'JobRole']:
    if c in filtered.columns:
        group_col = c
        break

# Use the renamed column if present
salary_col = None
for c in ['MonthlySalary', 'Salary', 'MonthlyIncome']:
    if c in filtered.columns:
        salary_col = c
        break

if group_col and salary_col:
```

```
    result = (
        filtered.groupBy(group_col)
        .agg(
            F.count(F.lit(1)).alias('employee_count'),
            F.avg(F.col(salary_col)).alias('avg_salary')
        )
        .orderBy(F.desc('employee_count'))
    )
    result.show(50, truncate=False)

    # HAVING-like filter
    result_filtered = result.filter(F.col('avg_salary') > 50000)
    result_filtered.show(truncate=False)
else:
    print('Required columns for groupBy not found. group_col:', group_col, 'salary_col:', salary_col)
```

Out [13]:

```
Required columns for groupBy not found. group_col: None salary_col: None
```

In [14]:

```
# --- Save Outputs as CSV ---
# Spark writes CSV into folders; we remove the `.csv` suffix by slicing off last 4 chars.

output_clean_path = '../output/cleaned_data.csv'
output_grouped_path = '../output/grouped_results.csv'

(
    df_clean.coalesce(1)
    .write
    .mode('overwrite')
    .option('header', True)
    .csv(output_clean_path[:-4])
)

if 'result' in locals():
    (
        result.coalesce(1)
        .write
        .mode('overwrite')
        .option('header', True)
        .csv(output_grouped_path[:-4])
    )

print('Saved outputs to output/ (Spark writes folders).')
```

Out [14]:

```
Saved outputs to output/ (Spark writes folders).
```

In [15]:

```
spark.read.option("header", True).csv("../output/cleaned_data").show(5)
```

Out [15]:

```
+---------+-----------+---------+-----------+----+------+-----------+------------------------+----------+
|Education|JoiningYear|     City|PaymentTier| Age|Gender|EverBenched|ExperienceInCurrentDomain|LeaveOrNot|
+---------+-----------+---------+-----------+----+------+-----------+------------------------+----------+
|      PHD|       2012|New Delhi|          3|27.0|  Male|         No|                       5|         0|
|      PHD|       2018|Bangalore|          3|26.0|  Male|         No|                       4|         1|
|  Masters|       2017|     Pune|          2|24.0|Female|         No|                       2|         0|
|Bachelors|       2013|Bangalore|          3|28.0|Female|         No|                       4|         1|
|Bachelors|       2018|Bangalore|          3|26.0|  Male|        Yes|                       4|         1|
+---------+-----------+---------+-----------+----+------+-----------+------------------------+----------+
only showing top 5 rows
```

In [17]:

```
print("result" in locals())
```

Out [17]:

```
False
```